SOFT COMPUTING ASSIGNMENT -9

Perumalla Dharan
AP21110010201

Given a dataset containing students' marks across multiple courses, develop a KSOM-based model that clusters students based on their academic performance. Use students' marks dataset without considering class label.

```
import numpy as np
import pandas as pd

class SOM:
    def __init__(self, num_features, num_clusters):
        self.weights = np.random.rand(num_features, num_clusters)  # Initialize weights randomly

    def find_winner(self, sample):
        # Calculate the Euclidean distance from the sample to each weight vector
        distances = np.linalg.norm(self.weights - sample[:, np.newaxis], axis=0)
        return np.argmin(distances)  # Index of the closest weight vector (winning cluster)

    def update_weights(self, sample, winner_index, learning_rate):
        # Update the weights of the winning cluster
        self.weights[:, winner_index] += learning_rate * (sample - self.weights[:, winner_index])

def load_data(file_path):
    # Load data from CSV file
    data = pd.read_csv(file_path)
    return data
```

```
def main():
    # Load the training dataset
    train_file_path = r'E:\SRM\Soft Computing\Lab 9 - 15th
Oct\training_dataset_students(1000).csv'
    training_data = load_data(train_file_path)

    # Prepare the data: dropping non-numeric and unnecessary columns
    features = training_data.drop(columns=['st_id'],
errors='ignore').values

    num_features = features.shape[1]
    num_clusters = 2  # Two clusters: Pass and Fail

    # Initialize and train the Self-Organizing Map
    som = SOM(num_features, num_clusters)
    learning_rate = 0.6

    # Training process
    for epoch in range(100):  # Adjust the number of epochs as necessary
        for sample in features:
            winner_index = som.find_winner(sample)
            som.update_weights(sample, winner_index, learning_rate)
        learning_rate /= 1.01  # Decay the learning rate

    # Testing dataset
    test_file_path = r'E:\SRM\Soft Computing\Lab 9 - 15th
Oct\students_testing.csv'
    testing_data = load_data(test_file_path)
    test_features = testing_data.drop(columns=['st_id'],
errors='ignore').values

    # Classify test samples
    print("Student Cluster Assignments (Pass/Fail) in Testing Dataset:")
```

```
    for idx, test_sample in enumerate(test_features):
        winner_index = som.find_winner(test_sample)
        if winner_index == 0:
            result = "Pass"  # Cluster 0 for Pass
        else:
            # Check if the student fails based on the given condition
            if np.any(test_sample < 40):
                result = "Fail"  # Cluster 1 for Fail
            else:
                result = "Pass"

        print(f"Student {idx + 1} has {result}")

if __name__ == "__main__":
    main()
```

```
Student Cluster Assignments (Pass/Fail) in Testing Dataset:
Student 1 has Fail
Student 2 has Pass
Student 3 has Fail
Student 4 has Fail
Student 5 has Pass
Student 6 has Pass
Student 7 has Fail
Student 8 has Fail
Student 9 has Pass
Student 10 has Pass
Student 11 has Pass
```